Backend Report

Tour Uzbekistan - backend status and related integrations
Actual code state as of July 11, 2026

Backend folder	backend
Framework	NestJS + TypeScript
Database	PostgreSQL via Prisma
Auth	JWT access + refresh, bcrypt, roles CUSTOMER and PARTNER
API prefix and docs	/api/v1 and Swagger at /api/docs
Current seeded catalog	5 published tours plus content data for home, countries, services, news, why-us, leads ar

1. Infrastructure and bootstrap

The project is bootstrapped in NestJS and wired as a modular monolith. The startup layer centralizes environment loading, global validation, CORS, API prefixing and Swagger generation.

- ConfigModule is global and reads .env.
- Global prefix is set to /api/v1.
- CORS is enabled from CORS_ORIGIN, including comma-separated origins.
- ValidationPipe uses whitelist, transform and forbidNonWhitelisted.
- Swagger is exposed at /api/docs with bearer auth configured.
- Health endpoint is implemented at GET /api/v1/health.
- Docker Compose includes PostgreSQL and the backend service, with backend connecting to the postgres container through DATABASE_URL.

2. Core backend modules

AppModule imports the backend feature set as separate Nest modules around a shared Prisma layer.

- PrismaModule: shared PrismaService that connects on module init.
- AuthModule: partner registration, login, refresh token, JWT strategy and guards.
- HomeModule: aggregates data for the home page in one endpoint.
- CountriesModule, ServicesModule, NewsModule, WhyUsModule, ToursModule: public content APIs backed by PostgreSQL.
- LeadsModule: receives public lead forms and validates linked entities.
- BookingsModule: protected B2B booking flow for authenticated partners only.
- HealthModule: service status endpoint.

3. Authentication and authorization

Authentication is implemented for partner accounts only, without admin CMS flows. Tokens are generated in AuthService and refresh tokens are stored as hashes, not in plain text.

- Partner registration endpoint: POST /api/v1/auth/register/partner.
- Login endpoint: POST /api/v1/auth/login.
- Refresh endpoint: POST /api/v1/auth/refresh.

- Passwords are hashed with bcrypt.
- Access and refresh tokens are signed with separate secrets and expirations from .env.
- New partner users are created with role PARTNER and linked Partner entity data.
- JwtAuthGuard protects secured routes. OptionalJwtAuthGuard allows tours endpoints to react to auth presence without forcing login.
- Booking creation is explicitly limited to role PARTNER; other roles receive Forbidden.

4. Public and protected API surface

Below is the currently implemented surface visible from frontend and partner flows.

Module	Implemented endpoints
Health	GET /api/v1/health
Home	GET /api/v1/home
Countries	GET /api/v1/countries, GET /api/v1/countries/:slug
Services	GET /api/v1/services, GET /api/v1/services/:slug
Why us	GET /api/v1/why-us/categories, GET /api/v1/why-us/categories/:slug
News	GET /api/v1/news, GET /api/v1/news/:slug
Tours	GET /api/v1/tours, GET /api/v1/tours/:slug
Leads	POST /api/v1/leads
Bookings	POST /api/v1/bookings, GET /api/v1/bookings/me
Auth	POST /api/v1/auth/register/partner, POST /api/v1/auth/login, POST /api/v1/auth/refresh

5. Content behavior and business rules

- All content modules support locale selection with ru, en and uz.
- Countries, services, why-us categories, news and tours return only published records.
- Countries, services, news and tours return proper 404 when a slug is not found.
- Services and news support pagination.
- News is sorted by publishedAt DESC.
- Why-us uses Prisma includes instead of N+1 fetching when loading categories with facts.
- Home endpoint aggregates banners, featured countries, featured tours, featured services, why-us categories and latest news in one database-backed response.

6. Tours and pricing logic

ToursService contains the richest public domain logic in the backend.

- Tours support filters by country, duration range, comfort stars range, price range and search text.
- Search checks translated title, route and description.
- Tour responses include images, day-by-day program, transport info, hotel info and included services.
- Price visibility depends on authorization: for unauthenticated requests the backend hides priceFrom when showPriceToB2C is false.
- Tours controller uses OptionalJwtAuthGuard so the same endpoint can serve both public and partner-specific views.
- Seed data now contains 5 published tours with different duration, country, comfort and pricing characteristics, making filters and pagination more realistic.

7. Leads and bookings

The form layer is already wired to persistent storage and linked entities.

- Public lead creation saves name, email, phone, message, source page, language and optional links to tour, country and service.
- Lead validation checks related entity existence before writing references.
- Booking creation is available only to authenticated PARTNER users.
- When a booking is created, the backend saves a snapshot of the chosen tour: title, price, currency, day-by-day program, transport, hotels and included services.
- Partner users can retrieve only their own bookings through GET /api/v1/bookings/me.
- This separation means public interest and partner sales flow already persist into different domain models.

8. Prisma schema and relations

The schema models both content and commercial flows with localization tables per main entity.

- Main localized entities: User, Partner, Country, Tour, TourDay, TourImage, Service, News, WhyCategory, WhyFact, Lead and Booking.
- Country -> Tour is one-to-many.
- Tour -> TourDay and Tour -> TourImage are one-to-many.
- Partner -> User is used for partner-linked auth and B2B bookings.
- Lead optionally links to Tour, Country and Service.
- Booking optionally links to Tour and Country, and directly links to Partner.
- Translation tables use unique pairs like entityId + locale.
- Several flexible frontend-facing content blocks are stored as JSON in translation records, for example country TOC/sections and included/excluded arrays.

9. Seed data and environment setup

- Seed resets translations and parent entities in dependency-safe order before recreating content.
- Seed currently creates a partner, a partner user, featured countries, home banners, services, why-us facts, news, leads, bookings and 5 tours with translations.
- Package scripts include prisma:generate, prisma:migrate:dev, prisma:validate and db:seed.
- Docker Compose exposes PostgreSQL on port 5432 and backend on port 3000.
- Environment variables include DATABASE_URL, JWT secrets, token expirations, PORT and CORS_ORIGIN.

10. What this backend is already connected to

From the current codebase state, the backend is not isolated boilerplate: it is already tied to frontend behavior and partner flows.

- Frontend public pages consume home, countries, services, why-us, news and tours endpoints.
- Public contact and request forms submit into Leads.
- Partner registration and login are real and issue JWT tokens.
- Booking forms already depend on auth presence and create real bookings instead of mock payloads.
- Swagger acts as the current machine-readable API contract for manual testing and integration checks.

Key backend files referenced in this report: main.ts, app.module.ts, auth/*, tours/*, bookings/*, home/*, countries/*, services/*, news/*, why-us/*, leads/*, prisma/schema.prisma, prisma/seed.ts, docker-compose.yml, .env.example.